Історія діалогу Cursor — завдання html2json

Проєкт jito-intern-test-task. Повідомлення з 27–29 жовтня 2026. У файл увійшли репліки користувача й відповіді асистента. Виклики інструментів і службові інструкції не включені.

КОРИСТУВАЧ

Tuesday, Oct 27, 2026, 7:44 PM (UTC+2)

Проаналізуй мені всі файли і папки в проекті, крім rozetka.html, поясни всі пункти та створи мені план реалізації та питання, які менв потрібно буде задати,якщо я не розумітиму як зробити це

АСИСТЕНТ (CURSOR)

Спочатку розберу структуру проєкту і зміст файлів, окрім `rozetka.html`, щоб план відповідав саме цьому завданню.

У README є посилання на шаблон і секцію Expected Deliverables, якої в локальному файлі немає. Перевірю оригінал завдання, щоб план покрив усі вимоги.

Завдання навмисно не задає формат JSON і забороняє DOM-парсер. Зберу план реалізації з розбором кожного файлу, кроків парсера і питань, які варто ставити, якщо якийсь крок буде незрозумілий.

КОРИСТУВАЧ

Wednesday, Oct 28, 2026, 8:02 PM (UTC+2)

скидаю. тобі код, той що я написала , він не весь, проаналізуй його і нвпиши покроково що я маю далі робити, який файл створюбвати і над якою задачею працювати. І розпиши кожен крок функції, яку я маю зробити

constants.js

```
export const VOID_TAGS = {
    area: true,
    base: true,
    br: true,
    col: true,
    embed: true,
    hr: true,
    img: true,
    input: true,
    link: true,
    meta: true,
    param: true,
    source: true,
    track: true,
    wbr: true,
};

export const RAWTEXT_TAGS = {
    script: true,
```

```
    style: true,
    textarea: true,
    title: true,
};

export const ENTITIES = {
    amp: "&",
    less_then: "<",
    greater_then: ">",
    quot: "",
    apos: "'",
    non_breaking_space: " ",
    copy: "©",
    registered: "®",
    trademark: "™",
    horizontal_ellipsis: "…",
    mdash: "—",
    ndash: "–",
    lsquo: "‘",
    rsquo: "’",
    ldquo: "“",
    rdquo: "”",
};

export const HEADING_TAGS = ["h1", "h2", "h3", "h4", "h5", "h6" ];

export function isLetter(char) {
    return (char >= "a" && char <= "z") || (char >= "A" && char <= "Z");
}
```

entities.js

```
import {ENTITIES} from "./constants";

export function decodeEntities(text){
    const entitiesRegex = /&(#x[0-9a-f]+|#[0-9]+|[a-z][a-z0-9]+);/gi;

    return text.replace(entitiesRegex, (match, entity) => {
        try {
            if(entity.startsWith("#")) {
                const isHex = entity[1].toLowerCase() === "x";

                let codePoint = isHex ? parseInt(entity.slice(2, 16)) : parseInt(entity.slice(1, 10));

                const isNumValid = !isNaN(codePoint) && codePoint >= 0 && codePoint <= 0x10ffff;
                return isNumValid ? String.fromCodePoint(codePoint) : match;
            }

            const isHasEntities = Object.prototype.hasOwnProperty.call(ENTITIES, entity);
```

```
            return isHasEntities ? ENTITIES[entity] : match;
        } catch (error) {
            return match;
        }
    })
}
```

АСИСТЕНТ (CURSOR)

Спочатку гляну, чи ці файли вже є в проєкті, і в якому стані зараз `html2json.js`.

Since I'm asked for instructions rather than implementation, I should actually check the workspace to see the real state of the files before analyzing. I'm noticing a likely mismatch between the module export style used in the JS file and how it's being loaded as a plain script in the HTML.

I'm also spotting a problem with the entity map—keys like "less_then" and "greater_then" don't match real HTML entity names such as "lt" and "gt", so those need fixing along with the nbsp value.

Looking deeper at entities.js, there's a parseInt bug: hex parsing omits the radix argument, which will misread hex codepoints. There's also truncation in the decimal slicing, questionable handling of surrogate ranges and null codepoints, and a regex that should still work fine for standard entity names.

I also notice the ES import for ENTITIES is missing a .js extension, and HEADING_TAGS seems defined inconsistently compared to other structures. I should also flag that title/textarea are RCDATA elements needing entity decoding, while script/style are raw text and shouldn't decode entities — this distinction is missing. Since using a DOM parser isn't allowed, I should check the workspace for these files before writing up the answer in Ukrainian.

I should check whether that character on line 33 is actually a non-breaking space by grepping for it.

Нічого не змінював, вся робота лишається за тобою. Нижче: що не так у вже написаному, потім кроки по файлах, а в кінці розписані функції.

1. Головна проблема: `import` / `export`

Сторінка index.html підключає код так: `<script src="html2json.js"></script>`. Це звичайний скрипт, а не модуль. Через це:

- Якщо додати `import` у `html2json.js`, браузер видасть `SyntaxError`, і кнопки перестануть працювати.
- Імпорт `"./constants"` без `.js` у браузері теж не спрацює.
- Кнопки в `index.html` викликають `convertHtml2JsonAndSet()` через `onclick`, а функції з модулів для `onclick` не видно.
- У README написано, що реалізація має лежати в `html2json.js`.

Рішення: весь код має бути в одному файлі `html2json.js`, без `import` і `export`. Файли `parser/constants.js` і `parser/entities.js` лишаються чернетками. Коли виправиш їх, перенеси вміст у `html2json.js` розділами: константи, сутності, парсер, UI. Це можна зробити в кінці, але з самого початку пиши код без `export`/`import`.

2. Помилки у твоєму коді

`entities.js`

1. **`parseInt` без системи числення.** `parseInt("1F600")` повертає `1`, а не 128512. Для hex треба передавати основу 16. Зрізи `slice(2, 16)` і `slice(1, 10)` обрізають довгі числа і видають хибний результат, тож їх прибери:
```js
const codePoint = isHex
  ? parseInt(entity.slice(2), 16)
  : parseInt(entity.slice(1), 10);
```
2. **Некоректні кодові точки.** `&#0;` і сурогати (`0xD800`–`0xDFFF`) `String.fromCodePoint` пропускає. За специфікацією HTML їх замінюють на `"\uFFFD"`. Додай цю перевірку.
3. **Імпорт.** `"./constants"` треба замінити на `"./constants.js"`. Після злиття в один файл імпорт зникне.
4. **`try/catch`** після перевірки діапазону майже не потрібен. Можна залишити як страховку, але тоді коментар, чому він там.
5. **Швидкий вихід.** На початку додай `if (!text || text.indexOf("&") === -1) return text;`. Це прискорить великі файли.
6. **Назва `isHasEntities`** краще замінити на `hasEntity`.

`constants.js`

1. **Ключі `ENTITIES` не є справжніми іменами сутностей.** Зараз `<` і ` ` не декодуються, бо ключі називаються `less_then`, `greater_then`, `non_breaking_space`, `registered`, `trademark`, `horizontal_ellipsis`. У HTML імена такі: `lt`, `gt`, `nbsp`, `reg`, `trade`, `hellip`. Решта (`amp`, `quot`, `apos`, `copy`, `mdash`, `ndash`, `lsquo`, `rsquo`, `ldquo`, `rdquo`) названі правильно.
2. **`nbsp` зараз звичайний пробіл.** Пошук `\xA0` у файлі нічого не знайшов. Значення має бути `"\u00A0"`.
3. **Доповни словник** тим, що часто трапляється на сайтах: `laquo`, `raquo`, `bull`, `middot`, `times`, `euro`, `hellip`, `deg`, `plusmn`.
4. **Пастка з `VOID_TAGS[tag]`.** Якщо шукати так, то для `<constructor>` або `<toString>` результат буде truthy, бо ці ключі є в прототипі `Object`. Використай `Set`: `new Set([...])` і `VOID_TAGS.has(tag)`. Для `RAWTEXT_TAGS` те саме.
5. **`script` і `style` не те саме, що `textarea` і `title`.** У `script` і `style` сутності **не** декодуються. У `textarea` і `title` вони декодуються, але теги всередині не розбираються. Розділи їх на два набори або перевіряй за назвою.
6. **`HEADING_TAGS` поки ніде не використовується.** Він знадобиться в кроці «неявне закриття», коли `<h1>` закриває відкритий `<p>`. Якщо цей крок не робитимеш, видали масив.
7. **`isLetter`** написана правильно. Використовуй її, щоб відрізнити `<div` (тег) від `a < b` (звичайний текст).

3. Що робити далі, по порядку

#	Задача	Файл
1	Виправити константи і сутності, переписати без `export`/`import`	`parser/*.js`, далі `html2json.js`
2	Фабрики вузлів і допоміжні функції	`html2json.js`

3	Читання атрибутів і тегів	`html2json.js`
4	Коментарі, doctype, сирий текст, звичайний текст	`html2json.js`
5	Головний цикл зі стеком (`html2json`)	`html2json.js`
6	`showExample1`, `showExample2`, захист у `convertHtml2JsonAndSet`	`html2json.js`
7	Семпли	`html_samples/*.html` (новий)
8	Скрипт перевірки	`test.js` (новий, лише для тебе)
9	README і `ai_help/`	`README.md`, `ai_help/` (нова папка)

Правило для задач 3–5: кожна функція отримує `(html, pos)` і повертає нову позицію. Глобальних змінних для позиції немає.

4. Функції по кроках

Формат результату зафіксуй такий:

```js
{ type: "document", children: [...], errors: [] }
{ type: "element", tag: "div", attributes: { class: "a" }, children: [...] }
{ type: "text", content: "..." }
{ type: "comment", content: "..." }
{ type: "doctype", content: "html" }
```

Задача 2. Допоміжні функції
- `createElement(tag, attributes)` повертає `{type:"element", tag, attributes, children:[]}`. Такі самі `createText`, `createComment`, `createDoctype`.
- `isWhitespace(ch)` повертає true для пробілу, `\t`, `\n`, `\r`, `\f`.
- `isTagStart(html, pos)`: за `<` має йти літера (`isLetter`). Тоді це відкривний тег.
- `isCloseTagStart(html, pos)`: `</` і далі літера.
- Для об’єкта атрибутів використовуй `Object.create(null)`. Інакше атрибут `__proto__` зіпсує об’єкт.

Задача 3. Теги і атрибути

`readTagName(html, pos)`
1. Іди від `pos`, поки не зустрінеш пробіл, `/`, `>` або кінець рядка.
2. Зріж підрядок через `slice`, не додавай по символу.
3. Приведи до нижнього регістру.
4. Поверни `{ name, pos }`.

`readAttributes(html, pos)` повертає `{ attributes, selfClosing, pos }`. Цикл:
1. Пропусти пробіли.
2. Кінець рядка: зупинись і запиши помилку «тег не закритий».
3. `>`: `pos++` і зупинись.
4. `/`: якщо далі `>`, постав `selfClosing = true`, зсунься на 2 і зупинись. Інакше пропусти `/` і продовжуй.
5. Прочитай ім’я атрибута до пробілу, `=`, `>` або `/`, і зроби його нижнім регістром. Якщо перший символ `=`, візьми його в ім’я, щоб цикл просувався.
6. Пропусти пробіли. Якщо далі `=`, пропусти його і пробіли, потім прочитай значення:
   - у лапках `"` або `'`: знайди закриття через `indexOf`. Якщо його немає, значення йде до кінця

рядка;
   - без лапок: до пробілу або `>`.
7. Якщо `=` немає, атрибут булевий. Значення `""`.
8. Значення пропусти через `decodeEntities`.
9. Якщо такого атрибута ще немає, запиши його (дублікат: виграє перший).
10. **Захист від зациклення:** якщо `pos` після ітерації не змінився, зроби `pos++`.

`parseOpenTag(html, pos)` (на `pos` стоїть `<`)
1. `readTagName` з `pos + 1`.
2. `readAttributes` з отриманої позиції.
3. Поверни `{ element, selfClosing, pos }`.

`parseCloseTag(html, pos)` (на `pos` стоїть `</`)
1. `readTagName` з `pos + 2`.
2. Знайди `>` через `indexOf`. Якщо його немає, позиція дорівнює довжині рядка.
3. Поверни `{ name, pos: індекс + 1 }`.

Задача 4. Решта вузлів

`parseComment(html, pos)` (на `pos` стоїть `<!--`)
1. `end = html.indexOf("-->", pos + 4)`.
2. Якщо `-1`, вміст іде до кінця рядка, позиція дорівнює довжині, запиши помилку.
3. Інакше `content = slice(pos + 4, end)`, позиція `end + 3`.

`parseDeclaration(html, pos)` (`<!DOCTYPE ...>` або `<?xml ...?>`)
1. Знайди `>`.
2. Якщо після `<!` йде `doctype` (без урахування регістру), створи doctype, інакше comment.
3. Якщо `>` немає, бери до кінця рядка.

`readRawText(html, pos, tagName)` (для `script`, `style`, `textarea`, `title`)
1. Не роби `html.toLowerCase()` для всього рядка: деякі символи міняють довжину при зміні регістру, і всі індекси «попливуть».
2. Створи `new RegExp("</" + tagName + "(?=[\\s/>])", "gi")`. Постав `lastIndex = pos` і виклич `exec`.
3. Знайшовся збіг: текст це `slice(pos, match.index)`, позиція `match.index`. Сам закривний тег потім розбере головний цикл.
4. Не знайшовся: текст іде до кінця рядка.
5. Декодуй сутності лише для `textarea` і `title`.

`readText(html, pos)`
1. Починай з `i = pos`. Знаходь наступне `<` через `indexOf`.
2. Якщо це не початок тега, коментаря чи закриття (`isTagStart`, `</` із літерою, `<!`, `<?`), вважай `<` частиною тексту і шукай далі.
3. Зріж, декодуй, поверни `{ content, pos }`.
4. Правило пробілів вирішиш сама (питання нижче).

Задача 5. Головна функція

`html2json(htmlText)`
1. Якщо вхід не рядок: `null` і `undefined` стають `""`, інше пропусти через `String(...)`.

2. Прибери BOM `\uFEFF` на початку.
3. Створи `root = createDocument()`, `stack = [root]`, `errors = []`.
4. Обгорни основний цикл у `try/catch`. Що встигло розпарситись, те й повернеться.
5. Цикл `while (pos < html.length)`:
   - `<!--` викликає `parseComment`;
   - `<!` або `<?` викликає `parseDeclaration`;
   - `</` із літерою викликає `parseCloseTag` і `handleCloseTag`;
   - `<` із літерою викликає `parseOpenTag` і `handleOpenTag`;
   - інакше це `readText`.
   - Наприкінці: `if (newPos <= pos) newPos = pos + 1`.
6. **`handleOpenTag`**:
   - Не пускай глибину стека вище `MAX_DEPTH` (наприклад, 512): далі вузол додається як сусід, а не дитина. Це захищає `JSON.stringify` від `RangeError` на дуже глибокому HTML.
   - Додай елемент у `stack[stack.length - 1].children`.
   - Якщо тег із `VOID_TAGS` або `selfClosing`, у стек його не клади.
   - Якщо тег із `RAWTEXT_TAGS`, викличи `readRawText`, додай текст дитиною і теж не клади в стек.
   - Інакше `stack.push(element)`.
7. **`handleCloseTag(name)`**:
   - Шукай `name` у стеку знизу вгору, але корінь не чіпай.
   - Знайшла на індексі `k`: прибери зі стека всі елементи вище `k` разом із `k`. Ті, що вище `k`, запиши в `errors` як «закрито неявно».
   - Не знайшла: ігноруй, запиши в `errors` («зайвий закривний тег»).
8. **Необов'язково — неявне закриття:** відкриття `<li>` закриває відкритий `<li>`, а `<h1>`–`<h6>` або `<div>` закривають `<p>`. Тут знадобиться `HEADING_TAGS`.
9. Після циклу елементи, що лишились у стеку, записати в `errors` як незакриті.
10. Поверни `root` із полем `errors`.

Задача 6. UI
- `showExample1()` записує приклад у `document.getElementById("html").value`. Перший: нормальна сторінка з `head`, `body`, атрибутами, коментарем і сутностями.
- `showExample2()`: такий самий, але з битим HTML (незакритий `<div>`, зайве `</p>`).
- У `convertHtml2JsonAndSet` оберни `html2json` і `JSON.stringify` у `try/catch`. На помилку виводь повідомлення в поле, а не падай.
- Наприкінці файлу, щоб Node міг запустити тести, а браузер не ламався:
```js
if (typeof module !== "undefined" && module.exports) {
  module.exports = { html2json };
}
```

Задача 7. Семпли
Окремі файли в `html_samples/`:
- порожній файл і голий текст;
- вкладені блоки, `<br>`, `<img>`, `<input>`;
- атрибути в лапках, без лапок, булеві, дублікати;
- коментарі й doctype, зокрема незакритий `<!--`;
- сутності: `&`, `😀`, `&unknown;`, `&#0;`;
- `script`/`style` зі строкою `"</div>"` усередині;
- биті випадки: `<div`, `<a href="x`, `</p>` без відкриття, `<div><p>text</div>`;

- дуже глибока вкладеність і великий файл (повтори шматок багато разів).

Задача 8. `test.js` (запуск: `node test.js`)
1. Для кожного файлу з `html_samples/`: виклич `html2json`, перевір, що немає винятку і що `result.type === "document"`.
2. Для кожного семпла відріж його на кількох випадкових довжинах і перевір, що функція не падає. Це перевірка на обірваний вхід.
3. Перевір кілька малих очікуваних результатів, наприклад `<p>Hi & bye</p>`.
4. Заміряй час на великому файлі. Якщо він ростить швидше, ніж розмір, десь є квадратична операція.

5. Що рішити самій

Це твої рішення, у README їх треба пояснити:
1. Булевий атрибут: `""` чи `true`?
2. Пробільні текстові вузли між тегами: викидати чи зберігати?
3. `<div/>`: вважати закритим чи відкритим? Для `svg` корисніше закритий.
4. Поле `errors`: завжди чи лише коли є помилки?

Питання для мене, якщо застрягнеш, добре працюють у такому вигляді: «Ось моя `readAttributes` і ось ввід, що вона повертає і чому зациклилась?»

Confirmed the value uses a regular space rather than a non-breaking space, so that's the bug. Now I'll write up the answer in Ukrainian with a structured analysis, next steps, which file and task to create, and detailed instructions.

On architecture, since index.html loads a plain script without module support and uses inline onclick handlers, I'll recommend consolidating everything into a single html2json.js file rather than splitting across parser/ modules with ES imports, since that would break the current setup — simplest and safest is one unified file.

For Node compatibility, I'll add a conditional module.exports export at the end so tests can import html2json without breaking browser usage.

I'm planning the internal structure: node factory helpers, whitespace checking, tag name/attribute parsing, open/close tag handlers, comment/doctype readers, raw text and text parsing, then the tree-building logic with a stack and implied end-tag handling for elements like p and li, followed by the main html2json function with error handling and example/test scripts.

Rather than full code, I'll give her pseudo-code for the main parsing loop since this is a learning exercise, reserving actual code snippets only for fixing her specific bugs.

Looking at her entities.js bugs: the hex parsing is slicing incorrectly and should parse the full remaining string with base 16, decimal similarly needs the full digit string rather than a truncated slice; I note that using Number or parseInt handles very large values gracefully by returning Infinity, which gets rejected by range checks. I also need her to handle code point 0 and surrogate range 0xD800–0xDFFF by returning the replacement character U+FFFD per spec, fix the import path to constants.js, and ensure her named entity table uses real HTML entity names like lt, gt, nbsp, reg, trade, and hellip.

Checking further, I see nbsp currently decodes to a plain space instead of U+00A0, and common entities like lt and nbsp aren't being decoded at all -- I'll suggest adding more entities like mdash, laquo, raquo, euro, bull, and middot as optional extras. Other minor suggestions: the try/catch is likely unnecessary since the range check already prevents fromCodePoint from throwing, renaming isHasEntities to hasEntity for clarity, and adding an early return when the text contains no ampersand for performance. For constants.js, I'm thinking through how RAWTEXT_TAGS should distinguish RCDATA tags like textarea and title (where entities still decode) from script/style (where they don't), and whether HEADING_TAGS is actually used anywhere or should be removed if it's just for future auto-closing logic.

I'm catching a serious bug risk: using plain objects like VOID_TAGS as lookup tables means a tag named "constructor" would return a truthy Function reference, falsely matching as a void tag -- same danger applies to attribute objects where a "__proto__" key could cause prototype pollution. The fix is to use hasOwnProperty checks, a Set, or Object.create(null) instead of plain object literals, same safe pattern already used for ENTITIES.

Now I'm sketching out the overall roadmap: starting with the single-file decision, then fixing constants/entities, defining the node format, implementing functions in dependency order with their inputs and outputs, building the main parsing loop, adding UI examples, creating samples and a test script, and finishing with documentation.

I'm settling on a cursor convention where functions take the html string and current position, returning an updated value and position without mutating anything globally, with the rule that every loop iteration must advance pos to avoid infinite loops. Now I'm drafting the node constructors for elements, text, comments, and doctypes, a whitespace checker, a tag-name reader, and an attribute reader that steps through skipping whitespace, detecting self-closing and unclosed tags, and parsing each attribute name and value.

For attribute values I'm handling quoted strings (finding the closing quote or falling back to end of string), unquoted values terminated by whitespace or ">", entity decoding, and boolean attributes defaulting to an empty string for JSON consistency, with duplicates resolved by first-wins and a safety increment if position doesn't move. Now I'm putting together the open-tag parser that reads the name and attributes and returns the resulting element, and starting on the close-tag parser.

Still writing parser...

A safer approach is a case-insensitive regex search anchored at the current position, matching closing tags like `</tagName` followed by whitespace or `>`, since tag names come from our known set so matching stays safe regardless of casing.

For parsing text, I'm scanning forward to the next "<" character, but treating it as literal text rather than a tag start if it's not followed by a letter, slash, bang, or question mark — using the isLetter check to distinguish real markup from stray angle brackets.

I'm leaning toward dropping whitespace-only text nodes outside pre/textarea for a cleaner JSON output, keeping other text unmodified without trimming. For the tree builder, I'm sketching a stack-based approach starting with a document node, where appendChild pushes onto the current node's children and open tags trigger implied closing logic.

I'm working out an auto-close map for elements like li, p, dt, dd, tr, td, th, and option that implicitly close their predecessors, then handling element insertion—pushing non-void elements onto the stack, treating

rawtext elements specially, and deciding to honor self-closing `/>` syntax as actually closing the element (useful for SVG) rather than following HTML's normal ignoring of that syntax for non-void elements.

For closing tags, I need to search the stack from top downward for a matching name, implicitly closing and flagging errors for anything above it, ignoring stray closers like `</br>` or `</p>` with no match, and never popping the document root; at EOF, any unclosed elements just get left as-is with recorded errors. I'm also flagging a risk: deeply nested structures could cause JSON.stringify recursion to throw a RangeError, so I should wrap the conversion in try/catch and consider imposing a max depth limit similar to the 512 cap used by browser parsers.

For large files, I should avoid repeated string slicing which causes O(n²) behavior, favoring index-based scanning instead. For the main html2json entry point, I'm guarding against non-string input, stripping a BOM character, optionally normalizing line endings, wrapping the parsing loop in try/catch to capture errors, and scanning character by character to dispatch tag parsing versus text accumulation.

I need a progress guard so the position always advances even on malformed input, and I'm deciding whether the returned document object should always include an errors array or only when non-empty. For the wrapper functions, I should catch JSON.stringify failures too, and set up example buttons that populate the textarea with sample HTML—one clean, one deliberately broken—plus a Node test script that runs against sample files and checks for no exceptions, with some fuzzing for robustness.

I'm also noting the user writes in Ukrainian and prefers feminine address ("ти"), so I should respond that way, and I want to flag the module-loading issue concretely: `import` statements fail in classic scripts, relative imports need explicit `.js` extensions, and `file://` module loading is blocked by browsers—so html2json.js currently has no way to run properly in that context.

Once she adds the imports, I think everything will break unless all the logic lives directly in html2json.js, with the parser/ folder serving only as drafts to copy from, since the README's deliverable list only references html2json.js and extra files may not even get loaded by index.html. I want to be thorough with numbered tasks and headings, referencing specific line numbers in her existing files like entities.js and constants.js so she can find exactly what needs fixing.

I'm working out the exact line-by-line structure of entities.js to cite it accurately, tracing through the import, regex, try/catch, and return statements to pin down which line numbers correspond to the isHex check, codePoint assignment, and the final catch block fallback.

I'm going to stop trying to pin down exact line numbers and instead reference the existing code loosely with backticks, putting my proposed changes in markdown code blocks. Time to write the actual answer now, keeping it thorough but reasonably concise.

КОРИСТУВАЧ

Wednesday, Oct 28, 2026, 8:13 PM (UTC+2)

1. Рішення: весь код має бути в одному файлі html2json.js, без import і export. - це написано в рідмі?? чому не можна в різних файлах? Так же ж більш т=структурованішйи код і читабельніший ?
2. entities.js - 1 пункт - потріьно написати цю систему числення
3.

АСИСТЕНТ (CURSOR)

I realize I overstated the README's constraints — it only says the implementation should be in html2json.js, it doesn't explicitly forbid multiple files. The real limitation is technical: index.html only loads that one script as a classic script, so I need to think through options for still organizing code across multiple files within that constraint.

КОРИСТУВАЧ

Wednesday, Oct 28, 2026, 8:17 PM (UTC+2)

1. Рішення: весь код має бути в одному файлі html2json.js, без import і export. - це написано в рідмі?? чому не можна в різних файлах? Так же ж більш т=структурованішйи код і читабельніший ?
2. entities.js - 1 пункт - потріьно написати цю систему числення :?? якщо це прибрати, що замість цього,??const codePoint = isHex
    ? parseInt(entity.slice(2), 16)
    : parseInt(entity.slice(1), 10);

АСИСТЕНТ (CURSOR)

1. Чому один файл

Я висловився занадто категорично. У README **не написано**, що код має бути в одному файлі. Там сказано лише: «`html2json.js` - This file should contain your implementation of the html2json function». Тобто функція `html2json` має бути саме там. Заборони ділити код на файли немає. Висновок про один файл я зробив із технічної причини.

index.html підключає код так: `<script src="html2json.js"></script>`. Це звичайний скрипт, який бачить лише цей один файл. Якщо розбити код на модулі, є три варіанти:

Варіант	Що треба	Ризик
Усе в `html2json.js`	нічого	немає
Кілька файлів із `import`/`export`	змінити `index.html`: `<script type="module">`, а функції для кнопок виставити через `window.convertHtml2JsonAndSet = ...`	якщо відкрити `index.html` подвійним кліком (`file://`), браузер блокує модулі, і сторінка не працює. Рецензент може запустити свою копію `index.html`, і тоді твоя зміна пропаде
Кілька звичайних скриптів	додати кілька `<script src="...">` в `index.html`	теж потрібно змінювати `index.html` і триматись порядку підключення

README дозволяє лишити `index.html` без змін, і це безпечніше. Завдання перевіряють запуском у браузері, а за умовою код, що падає, означає провалену роботу. Тому я радив не ризикувати.

Читабельності це не заважає. В одному файлі можна зробити чіткі розділи:

```js
// ===== Constants =====
// ===== Entities =====
// ===== Node factories =====
// ===== Tag and attribute parsing =====
// ===== Tree builder =====
// ===== html2json (public) =====
// ===== UI helpers =====
```

```

Якщо ти все одно хочеш кілька файлів, це можливо. Тоді скажи, який варіант із таблиці обираєш, і я розпишу кроки. Найнадійніше було б перевірити це в команди, що дала завдання, але для цього я не можу їм написати.

## 2. Що з `parseInt`

Ти зрозуміла навпаки: цей код **не додається** до твого, а **замінює** рядок із `parseInt`. Число `16` і `10` не треба «писати окремо». Це другий аргумент `parseInt`, система числення: 16 означає шістнадцяткову, 10 десяткову.

У тебе зараз:

```js
let codePoint = isHex ? parseInt(entity.slice(2, 16)) : parseInt(entity.slice(1, 10));
```

Має бути:

```js
const codePoint = isHex
  ? parseInt(entity.slice(2), 16)
  : parseInt(entity.slice(1), 10);
```

Що змінилось:

1. **Другий аргумент `parseInt`.** Без нього `parseInt("1F600")` читає число як десяткове, доходить до `F` і зупиняється, тому повертає `1`. З `16` повертається `128512`, тобто 😀.
2. **`slice(2, 16)` стало `slice(2)`.** У твоєму коді числа `2, 16` і `1, 10` були межами зрізу рядка, а не системою числення. Це схоже на іншу річ, тому легко сплутати. Зріз `slice(2, 16)` обрізав би рядок після 14 символів, а `slice(1, 10)` після 9. Для довгих чисел це дає хибний результат. Достатньо `slice(2)` (від 2-го символу до кінця) і `slice(1)`.
3. **Для `😀`** `entity` дорівнює `"#x1F600"`. `slice(2)` відкидає `#x` і лишає `"1F600"`. Для `©` `entity` це `"#169"`, `slice(1)` відкидає `#` і лишає `"169"`.

Решту функції (перевірку `isNumValid`, `String.fromCodePoint`) лишай, лише додай перевірку на `0` і діапазон суррогатів `0xD800`–`0xDFFF`, як я писав раніше. В обох випадках результатом має бути `"\uFFFD"`.

I'm reconsidering whether the README truly requires a single-file implementation, since index.html only loads html2json.js as a classic script—meaning modules or multiple scripts could break compatibility with file:// access or existing onclick handlers, and reviewers might swap in their own index.html anyway.

For the second issue, I realize she misunderstood the parseInt lines: the "10" and "16" are just radix arguments, not new instructions to follow separately. I should show the old code versus the corrected version, explaining that without an explicit radix, parseInt("1F600") stops parsing at "F" and returns 1, and the "#x" prefix needs slice(2) plus radix 16 to parse correctly as hex.
```

КОРИСТУВАЧ

Wednesday, Oct 28, 2026, 9:25 PM (UTC+2)

я вирішую ті задачі, які ти сворив мені, перевіряй їх , якщо є помилка, поясни, але не пиши відповідь поки я не попрошу

АСИСТЕНТ (CURSOR)

Добре. Правила такі:

- Ти пишеш код, я його перевіряю.
- Якщо знаходжу помилку, пояснюю, що не так і чому. Виправлений код не показую, поки ти не попросиш.
- Можеш просто написати «перевір задачу 3». Я сам прочитаю файли з проєкту, вставляти код у чат не обов'язково.
- Файли в проєкті я не змінюю.

Коли будеш готова, пиши, яку задачу перевірити.

She's saying she'll check her tasks and I should review them later, pointing out errors without giving away answers until she asks. Since no code has been submitted yet, I should just acknowledge this and wait for her to share a specific task or file to look at.

КОРИСТУВАЧ

Wednesday, Oct 28, 2026, 9:54 PM (UTC+2)

перевір задачу 2, але осьанній пункт задачі я не розумію

АСИСТЕНТ (CURSOR)

Спочатку подивлюся, що ти вже написала.

She's stuck on the last point about using Object.create(null) for the attributes object, so that __proto__ doesn't corrupt it — I should explain the reasoning behind this rather than just hand her the fix, and I also need to look at her task 2 code files to see what she's actually written.

I should focus on reviewing for errors and explaining them conceptually rather than handing over fixed code, and I also want to explain Object.create(null) since she asked about it. First issue I'm spotting: using export in what looks like a classic script context would throw a SyntaxError and break the whole file, plus there's still the underlying object-vs-Set prototype concern to flag.

I'm also catching a typo where "lh" should be "lt" for the less-than entity, and I need to verify whether nbsp maps to the correct U+00A0 codepoint. There's a more serious logic bug in decodeEntities where the validity condition is inverted, making only surrogate pairs count as valid and letting normal characters fall through incorrectly — I want to explain why it should instead exclude surrogates and null, replacing them with the replacement character, without just handing over corrected code. I'm also noting that createText, createComment, and createDoctype look like copy-pasted versions of createElement with the wrong parameters and type labels, and that isTagStart/isCloseTagStart are actually safe even when indexing past the string end since comparing undefined with a string just evaluates to false. Everything else — HEADING_TAGS, whitespace handling, isLetter, the try/catch, and the entity lowercase call — looks fine so far, and I'm continuing into the RAWTEXT decoding section next.

Помилки в задачі 2

Задачу вже перенесено в `html2json.js`, перевіряю там.

1. `export` зламає весь файл.
Рядок `export const VOID_TAGS = {` лишився з модульної версії. У звичайному `<script>` це `SyntaxError`, тож не працюватиме нічого, включно з кнопкою. Прибери слово `export`.

2. Одрук у ключі сутності.
`lh: "<"` має бути `lt`. Через це `<` не декодується.

3. `nbsp` досі звичайний пробіл.
Пошук символу U+00A0 нічого не знайшов. Звичайний пробіл у JS можна записати як `" "`, а нерозривний пробіл треба записати escape-послідовністю, бо в редакторі їх не відрізнити. Знайди, як записується символ з кодом `00A0` у рядку JS.

4. Перевірка в `decodeEntities` інвертована.
Ось ця умова:

```js
codePoint >= 0xd800 && codePoint <= 0xdfff && codePoint <= 0x10ffff
```

Вимагає, щоб число було суррогатом (`0xD800`–`0xDFFF`). Для `A` (літера A) вона дає false, і функція поверне `A` без змін. Вийшло навпаки.

Логіка має бути така:
- число не `NaN` і не більше `0x10ffff`: інакше залишаємо `match`;
- число `0` або суррогат: це недопустимі значення, замість них треба `"\uFFFD"`, а не залишати `match`;
- решта: `String.fromCodePoint`.

Поміркуй, як записати це двома окремими `if`. Умова `codePoint >= 0` зайва: регулярний вираз не пропускає мінус.

5. `createText`, `createComment`, `createDoctype` скопійовані з `createElement`.
Зараз усі чотири функції повертають однаковий об'єкт, і `type` у кожного `"elements"`. Треба:
- `type` у `createElement` має бути `"element"` (однина), як у форматі, який ми узгодили;
- у `createText`, `createComment`, `createDoctype` параметри `tag` і `attributes` не потрібні. Вони приймають текст і повертають вузол із `type: "text"`, `"comment"` або `"doctype"` і полем `content`. Масив `children` їм не потрібен.

6. Що в порядку.
`isLetter`, `isWhitespace`, `isTagStart`, `isCloseTagStart` написані правильно. Для кінця рядка, де `html[idx + 1]` це `undefined`, `isLetter(undefined)` просто повертає `false`, нічого не падає. Перевірка `hasOwnProperty` для `ENTITIES` теж правильна.

Одне зауваження про стиль: `RAWTEXT_DEC_TAGS` складно зрозуміти з назви. Ці теги в

специфікації називаються RCDATA, можна назвати так.

Останній пункт задачі 2: `Object.create(null)`

Це пояснення тому, чому так пишуть. Код вирішиш писати сама.

У JavaScript кожен звичайний об’єкт створюється з **прототипом**. Це схований «батьківський» об’єкт, звідки береться все, чого немає у твоєму. Подивись:

```js
const attrs = {};
attrs.constructor;   // не undefined! Це функція Object, береться з прототипа
attrs.toString;      // теж є
attrs["__proto__"];  // доступ до самого прототипа
```

Тож у звичайному `{}` вже ніби лежать ключі `constructor`, `toString`, `__proto__`, яких ти не додавала.

Чому це небезпечно для нашого парсера. Нехай в HTML є такий тег:

```html
<div __proto__="x" constructor="y">
```

Ми розбираємо атрибути й пишемо `attributes[name] = value`:
- якщо `name === "__proto__"`, запис у звичайний об’єкт не створює ключ, а змінює прототип. Атрибут губиться, а об’єкт поводиться дивно;
- перевірка «чи такий атрибут уже є» (`if (attributes[name])`) повертає `true` для `constructor` ще до того, як ти його додала. Атрибут `constructor` би просто пропустився.

`Object.create(null)` створює об’єкт **без прототипа**. У ньому немає нічого, крім того, що ти сама поклала. Тоді `attributes["constructor"]` це `undefined`, поки його не додали, а `__proto__` стає звичайним ключем. Вивід `JSON.stringify` не змінюється.

Те саме стосується `VOID_TAGS`. Для `<constructor>` вираз `VOID_TAGS["constructor"]` truthy, і парсер вважає цей тег порожнім. Тому раніше я радив `Set` або перевірку через `hasOwnProperty` (як ти вже зробила для `ENTITIES`).

Де це застосувати в задачі 2: у `createElement`, замість `attributes || {}`. Подумай, як записати «якщо атрибути не передали, візьми порожній об’єкт без прототипа».

Виправ пункти 1–5 і напиши, коли перевіряти знову.

That's just a regular space character, not something unusual. Now I'll write the review, explaining the concept of Object.create(null) with a small illustrative demo rather than handing over a fixed solution.

КОРИСТУВАЧ

Thursday, Oct 29, 2026, 9:07 PM (UTC+2)

1. передивись що я написала і підкажт що не так
2. допоможи написати головну функцію

АСИСТЕНТ (CURSOR)

1. Що не так у твоєму коді

Рядок `decodeEntities` і блок `readAttributes` мають найсерйозніші помилки. Далі йду від найважливіших до дрібних.

Критичні (зависання або падіння)

`readAttributes`: нескінченний цикл. Це найнебезпечніша помилка.
```js
while (idx < lengthHtml) {
  const startIdx = idx;
  idx = skipWhitespace(idx);
}
```
Після `skipWhitespace` нічого не змінюється, умова лишається істинною, цикл не завершується, вкладка зависає. Закриваюча `}` стоїть занадто рано. Вона має закінчувати цикл лише після **всіх** кроків (ім'я, `=`, значення, запис в об'єкт). Зараз усе це лежить поза циклом, тож навіть без зависання розбирався б максимум один атрибут. Захист від зациклення (закоментований наприкінці) теж має бути всередині цього циклу, бо `startIdx` існує тільки там.

Порожній `return;` у `readTagName`, `readAttributes`, `readText`. У JavaScript `return;` без значення повертає `undefined`. Ти хотіла вийти з циклу, а вийшла з усієї функції. Наприклад, у `readTagName` тег `<div>` ніколи не дійде до рядка `return { tagName, idx }`, бо функція виходить раніше. Далі `tagResult.idx` кине `TypeError`. Для виходу лише з циклу є `break`. Те саме в `readAttributes` (випадки `>`, `/>`, кінець рядка, кінець імені атрибута) і двічі в `readText`. У `readAttributes` після виходу з циклу має виконатись `return { attributes, selfClosing, idx }`.

Різні назви поля. `readTagName` повертає `tagName`, а `parseOpenTag` і `parseCloseTag` читають `tagResult.name`. Це завжди `undefined`.

Помилки логіки

`decodeEntities`: змінна `codePoint` оголошена всередині блоку `if (entity.startsWith("#"))`, а перевірка `isSurrogate` стоїть **поза** ним. Там `codePoint` не існує, виникає `ReferenceError`. Через `try/catch` ця помилка нікому не видна, і функція мовчки повертає `match`. У результаті жодна **іменована** сутність (`&`, `<`) не декодується. Перевірку на `0` і сурогат треба перенести всередину блоку, до `isNumValid`. Висновок: `try/catch` ховає такі помилки, тому під час розробки його краще прибрати, а потім вирішити, чи потрібен він.

`createComment`: `type: "content"` має бути `"comment"`.

`parseOpenTag`: створює об'єкт вручну (`name`, без `children`, без `tag`). У тебе є `createElement`, використай його. Інакше в елементів не буде масиву `children`, і головна функція впаде на `.children.push`.

`readText`: після виправлення `return;` на `break` коду, що йде після циклу, достатньо. Також для закривного тега в тебе є готова `isCloseTagStart`, а ти знову пишеш його перевірку вручну. Користуйся нею.

Менш критичне, але треба виправити

- **Послідовність імен.** `parseComment` і `parseDeclaration` повертають `pos`, решта `idx`. Лиши один варіант, `idx`.
- **`typeof decodeEntities === "function"`** (у `readAttributes`, `readRawText`, `readText`) зайве. Функція оголошена в цьому ж файлі, вона завжди існує. Прибери.
- **Порівняння з пробілами в п'яти місцях** (`char === " " || char === "\t" ...`). У тебе вже є `isWhitespace`, і `skipWhitespace` теж можна написати через неї.
- **`parseDeclaration`** кладе в `content` весь текст `<!DOCTYPE html>`, а у форматі було `"html"`. Вирішуй сама, але якщо лишаєш так, це має бути свідоме рішення, і про нього треба написати в README. До речі, у тебе вже є `createComment` і `createDoctype`, використай їх тут і в `parseComment`.
- **`createElement`** досі має `attributes || {}`. Те, про що ми говорили з `Object.create(null)`, ще не зроблено. У `readAttributes` теж `const attributes = {}`.
- **Через `VOID_TAGS[tag]`:** для `<constructor>` це значення truthy. Для перевірки зроби функцію `hasOwn(obj, key)` через `Object.prototype.hasOwnProperty.call` (ти вже робиш це в `decodeEntities`) або заміни об'єкти на `Set`.
- **`console.log`/`console.error`** усередині помічних функцій. Краще повертати текст помилки й збирати її в головній функції. Нижче так і зроблено.

Рекомендую виправити все це, а потім скопіювати головну функцію.

2. Головна функція

Нижче три частини: підготовка входу, диспетчер `parseNext`, робота зі стеком. Вони розраховані на те, що ти виправиш помічні функції так, щоб вони повертали:

Функція	Повертає
`readTagName`	`{ tagName, idx }`
`parseOpenTag`	`{ element, selfClosing, idx }`, де `element` створено через `createElement(tagName, attributes)`
`parseCloseTag`	`{ tagName, idx }`
`parseComment`, `parseDeclaration`	`{ node, idx }`
`readRawText`, `readText`	`{ content, idx }`

```js
const MAX_DEPTH = 512;
const MAX_ERRORS = 100;

function hasOwn(obj, key) {
  return Object.prototype.hasOwnProperty.call(obj, key);
}

function reportError(errors, message) {
```

```
  if (errors.length < MAX_ERRORS) {
    errors.push(message);
  }
}

function prepareInput(value) {
  if (value === null || value === undefined) {
    return "";
  }
  const text = typeof value === "string" ? value : String(value);
  return text.charCodeAt(0) === 0xfeff ? text.slice(1) : text;
}

function isBlank(text) {
  for (let i = 0; i < text.length; i++) {
    if (!isWhitespace(text[i])) {
      return false;
    }
  }
  return true;
}

function closeElement(stack, tagName, errors) {
  for (let i = stack.length - 1; i > 0; i--) {
    if (stack[i].tag === tagName) {
      for (let j = stack.length - 1; j > i; j--) {
        reportError(errors, `<${stack[j].tag}> closed implicitly by </${tagName}>`);
      }
      stack.length = i;
      return;
    }
  }
  reportError(errors, `Stray closing tag </${tagName}>`);
}

function openElement(html, stack, parsed, errors) {
  const { element, selfClosing } = parsed;
  let idx = parsed.idx;

  stack[stack.length - 1].children.push(element);

  if (selfClosing || hasOwn(VOID_TAGS, element.tag)) {
    return idx;
  }

  if (stack.length >= MAX_DEPTH) {
    reportError(errors, `Maximum depth exceeded at <${element.tag}>`);
    return idx;
  }
```

```
  stack.push(element);

  if (hasOwn(RAWTEXT_TAGS, element.tag)) {
    const raw = readRawText(html, idx, element.tag);
    if (raw.content !== "") {
      element.children.push(createText(raw.content));
    }
    idx = raw.idx;
  }

  return idx;
}

function parseNext(html, idx, stack, errors) {
  const parent = stack[stack.length - 1];

  if (html.startsWith("<!--", idx)) {
    const result = parseComment(html, idx);
    parent.children.push(result.node);
    return result.idx;
  }

  if (html[idx] === "<" && (html[idx + 1] === "!" || html[idx + 1] === "?")) {
    const result = parseDeclaration(html, idx);
    parent.children.push(result.node);
    return result.idx;
  }

  if (isCloseTagStart(html, idx)) {
    const result = parseCloseTag(html, idx);
    closeElement(stack, result.tagName, errors);
    return result.idx;
  }

  if (isTagStart(html, idx)) {
    return openElement(html, stack, parseOpenTag(html, idx), errors);
  }

  const result = readText(html, idx);
  if (!isBlank(result.content)) {
    parent.children.push(createText(result.content));
  }
  return result.idx;
}

function html2json(htmlText) {
  const root = { type: "document", children: [], errors: [] };
  const stack = [root];
```

```
  try {
    const html = prepareInput(htmlText);
    let idx = 0;

    while (idx < html.length) {
      const nextIdx = parseNext(html, idx, stack, root.errors);
      idx = nextIdx > idx ? nextIdx : idx + 1;
    }

    for (let i = stack.length - 1; i > 0; i--) {
      reportError(root.errors, `Unclosed tag <${stack[i].tag}>`);
    }
  } catch (error) {
    reportError(root.errors, `Unexpected error: ${error.message}`);
  }

  return root;
}
```

Як це працює, по кроках

1. **`html2json`** створює корінь і стек. У стеку завжди лежить поточний «відкритий» елемент, і нові вузли додаються в його `children`.
2. **Цикл `while`.** Кожну ітерацію `parseNext` обробляє один шматок і повертає нову позицію. Рядок `nextIdx > idx ? nextIdx : idx + 1` гарантує, що цикл завжди просувається, навіть якщо якась функція помилилась. Нескінченного циклу не буде.
3. **`parseNext`** дивиться, що починається на `idx`, у порядку: коментар, `<!`/`<?`, закривний тег, відкривний тег, інакше текст. Порядок важливий: `<!--` треба перевіряти раніше за `<!`.
4. **`openElement`.** Додає елемент у поточного батька. Порожні теги й `<tag/>` у стек не потрапляють. Інші потрапляють, а для `script`/`style`/`textarea`/`title` ще й читається сирий вміст. Закривний тег потім заверне їх звичайним шляхом.
5. **`closeElement`.** Шукає тег у стеку знизу вгору. Знайшла: усе вище нього закривається неявно (`<div><p>text</div>` закриває `p` і `div`). Не знайшла: це зайвий закривний тег, його ігноруємо. Корінь (`i > 0`) не чіпаємо.
6. **`MAX_DEPTH`** захищає `JSON.stringify` від `RangeError` на дуже глибокому HTML. `MAX_ERRORS` не дає масиву помилок розростись на гігантських зламаних файлах.
7. **`try/catch` навколо всього** гарантує вимогу «не падати». Те, що встигли розібрати, повернеться з полем `errors`.

Рішення, які лишаються за тобою

- Порожні рядки пробілів між тегами зараз відкидаються (`isBlank`). Якщо хочеш їх зберігати, прибери цю перевірку, але тоді JSON стане набагато довшим.
- Неявне закриття `<li>` і `<p>` (де знадобився б `HEADING_TAGS`) не зроблено. Спершу домогтись, щоб усе працювало, а потім за бажанням додати.

Коли виправиш помічні функції й вставиш головну, напиши, і я перевірю. Тоді ж можна братися за

`showExample1`/`showExample2` і тестовий скрипт.

She's asking two things now: point out what's wrong in her code, and help her write the main function. Since she's explicitly requesting help this time, I should guide her toward a solution with structure and hints rather than just handing over the full corrected code outright.

Going through her decodeEntities function, I'm spotting a scoping bug — she declares a variable inside the numeric-entity branch but then references it outside that block, which would throw a ReferenceError that gets silently swallowed by the try/catch, breaking named-entity handling too.

In readTagName there's a return statement where there should be a break, plus inconsistent property naming between what's returned and what parseOpenTag expects, and the resulting element object is missing children and uses the wrong key for the tag name. I'll note these along with smaller cleanup items like using Object.create(null) for attributes and avoiding plain objects for VOID_TAGS.

Looking at readAttributes next, I spot a genuinely broken loop structure: the whitespace-skipping while loop never advances past the check and will hang forever, and the closing brace for the outer loop is placed too early so only one attribute ever gets parsed instead of looping through all of them. There are also several bare `return;` statements that should instead break the loop and return the proper {attributes, selfClosing, idx} object — as written, parseOpenTag will crash on undefined.

Still spotting more issues... the stray-case branch needs a `continue`, the redundant `typeof decodeEntities` checks should be removed, using `Object.create(null)` for attributes is fine with `hasOwnProperty.call`, and the progress guard needs to live inside the loop rather than being computed once before it.

In parseOpenTag and parseCloseTag, both reference `tagResult.name` when the actual property is `tagName`, which would crash or silently produce undefined elements — these need fixing to use the right property and to guard against an undefined tagResult. Also noticing parseComment returns `{node, pos}` while sibling functions return a plain `idx`, which is an inconsistent naming convention worth unifying, and parseDeclaration's handling of raw content up to the closing `>` seems okay but worth double-checking edge cases.

For the doctype case, I'm weighing whether content should just be "html" versus the full raw `<!DOCTYPE html>` tag, and thinking it might be cleaner to strip the `<!doctype` prefix and trailing `>` for consistency, though this is more of a design choice than a bug. The RegExp-based search for closing tags in readRawText looks fine with its global flag and lastIndex tracking, and entity decoding for textarea/title could leverage the existing RCDATA_TAGS set rather than duplicating logic — the `lowerTag` variable also seems redundant there.

In readText, I'm spotting a real bug: there are bare `return;` statements inside a loop that should be `break` instead, since returning early would exit the whole function with undefined rather than just stopping the loop.

The code after the loop that slices out the text content is actually unreachable since every exit path currently returns early, meaning the loop body never finishes naturally — so that whole section is broken. I'm also noting that the isCloseTag check duplicates logic that already exists via isCloseTagStart, and should just reuse that helper for cleanliness. Walking through edge cases like a lone `<` in plain text, or sequences like `</>` or `</` followed by a non-letter, the loop still makes forward progress since the index

gets incremented when the angle bracket isn't a real tag start — so no infinite loop there, since `<!` and `<? ` are already intercepted by the main dispatcher before readText is ever called.

Looking further at html2json, the null-input handling is incomplete — it should return a proper empty document structure with children and errors arrays rather than bailing out early, and the try/catch currently only wraps the conversion step while the function itself doesn't return anything meaningful. I also notice unused constants like HEADING_TAGS alongside RAWTEXT_TAGS and RCDATA_TAGS which are defined but need checking, and I'm thinking main should use the createElement helper with tag and attributes rather than constructing elements manually. For error handling, I'm considering whether helpers should return error messages back to main to collect into an errors array instead of just logging to console.

Since she's now directly asking for help writing the main function, I'm leaning toward providing an actual code skeleton that uses the helper contract I've been designing, unifying field naming to use "idx" consistently and removing console calls from helpers to keep things clean. I'll give real working code for the main parsing loop with stack-based tag handling, since she's explicitly requesting this now rather than me volunteering it unprompted.

Still writing parser loop...

Finishing prepareInput by converting the value to a string and stripping a leading BOM character if present. I'm also noting that String(value) could throw for symbols or objects with bad toString methods, but that's covered by the outer try/catch.

Now I'm writing parseNext, which looks at the current position in the HTML and dispatches to the right parser—checking for comments first, then declarations/processing instructions, pushing the resulting node onto the parent's children and returning the updated index.

Continuing dispatch logic...

I should decide whether to use trim() for the whitespace check, but I need to be careful since trimming could strip decoded non-breaking space entities too, potentially losing meaningful content.

Writing the openElement function...

I'm thinking through edge cases: a depth guard that stops pushing deeply nested elements onto the stack to avoid stack overflow, though this could cause closing tags to mismatch ancestors in pathological cases, and handling raw text tags like script/style that need special parsing until their matching close tag.

I realize it's cleaner to push rawtext elements onto the stack normally, read their raw content separately, then let the natural closing tag pop them off, rather than trying to handle closure differently -- self-closing variants just skip the raw read entirely.

For void tag lookups I should use a safe hasOwnProperty helper to avoid prototype pollution issues, reusing it for entity decoding too. I'll defer implied end tag handling (like for `li` or `p`) to a later step, and now I'm writing the closeElement function that walks the stack backward to find a matching tag, logging implicit closures for anything nested inside before truncating the stack to that point.

I also want to guard against pathological inputs where huge numbers of stray closing tags could flood the errors array with a million messages, so I'm adding a capped error reporter (maybe 100 max) and routing

all error pushes through it for robustness.

Now I'm checking the stack element stores the correct tag reference and that `errors` only live on root, confirming the infinite-loop guard and recursion depth limits are sound, and noting a missing `Object.create(null)` detail in createElement to flag.

I'm planning to structure my answer in Ukrainian: first a prioritized list of issues (crashes/hangs first, then logic, then style) without fixed code, then the main function implementation along with the contract her helper functions need to satisfy, like how parseOpenTag should return results compatible with createElement.

КОРИСТУВАЧ
Thursday, Oct 29, 2026, 9:26 PM (UTC+2)

напиши мені тут де і що саме я маю додати

АСИСТЕНТ (CURSOR)

Спершу гляну поточний стан файлу, щоб вказати точні місця.

She needs exact placement instructions, so I should re-read the file first to check its current state in case it's changed.

Файл після останньої перевірки не змінився, тож усе, що я писав раніше, досі актуально. Нижче кроки по порядку: що замінити, де це розташоване і чим замінити. Номери рядків орієнтовні, шукай функцію за назвою. **Замінюй цілу функцію**, а не окремі рядки: так простіше не пропустити зайву дужку.

Крок 1. Верх файлу: видали закоментований `html2json`
Рядки ≈ 9–16 (коментар «Update this function...» і закоментована заглушка). Вони більше не потрібні.

Крок 2. `decodeEntities` (≈ 76–107): заміни цілу функцію
```js
function decodeEntities(text) {
  if (!text || text.indexOf("&") === -1) {
    return text;
  }

  const entitiesRegex = /&(#x[0-9a-f]+|#[0-9]+|[a-z][a-z0-9]+);/gi;

  return text.replace(entitiesRegex, (match, entity) => {
    if (entity[0] === "#") {
      const isHex = entity[1] === "x" || entity[1] === "X";
      const codePoint = isHex
        ? parseInt(entity.slice(2), 16)
        : parseInt(entity.slice(1), 10);

      if (isNaN(codePoint) || codePoint > 0x10ffff) {
        return match;
      }
```

```
      const isSurrogate = codePoint >= 0xd800 && codePoint <= 0xdfff;
      if (codePoint === 0 || isSurrogate) {
        return "\uFFFD";
      }

      return String.fromCodePoint(codePoint);
    }

    return hasOwn(ENTITIES, entity) ? ENTITIES[entity] : match;
  });
}
```
Що змінилось: перевірка на `0` і сурогат тепер усередині блоку, де існує `codePoint`. `try/catch` прибрано: після перевірок діапазону `fromCodePoint` не кидає винятків, а `catch` лише ховав помилки.

Крок 3. Фабрики вузлів (≈ 110–137)
- `createElement`: рядок `attributes: attributes || {},` заміни на `attributes: attributes || Object.create(null),`
- `createComment`: `type: "content"` заміни на `type: "comment"`.

Крок 4. `readTagName` (≈ 169–184): заміни цілу функцію
```js
function readTagName(html, idx) {
  const start = idx;

  while (idx < html.length) {
    const char = html[idx];
    if (isWhitespace(char) || char === "/" || char === ">") {
      break;
    }
    idx++;
  }

  return { tagName: html.slice(start, idx).toLowerCase(), idx };
}
```

Крок 5. `readAttributes` (≈ 186–292): заміни цілу функцію
Разом із закоментованим «хвостом» наприкінці.
```js
function readAttributes(html, idx) {
  const length = html.length;
  const attributes = Object.create(null);
  let selfClosing = false;

  while (idx < length) {
    const loopStart = idx;
```

```
while (idx < length && isWhitespace(html[idx])) {
  idx++;
}
if (idx >= length) {
  break;
}

const char = html[idx];

if (char === ">") {
  idx++;
  break;
}

if (char === "/") {
  if (html[idx + 1] === ">") {
    selfClosing = true;
    idx += 2;
    break;
  }
  idx++;
  continue;
}

const nameStart = idx;
if (html[idx] === "=") {
  idx++;
}
while (idx < length) {
  const c = html[idx];
  if (isWhitespace(c) || c === "=" || c === ">" || c === "/") {
    break;
  }
  idx++;
}
const name = html.slice(nameStart, idx).toLowerCase();

while (idx < length && isWhitespace(html[idx])) {
  idx++;
}

let value = "";
if (html[idx] === "=") {
  idx++;
  while (idx < length && isWhitespace(html[idx])) {
    idx++;
  }

  if (idx < length) {
```

```
        const quote = html[idx];
        if (quote === "" || quote === """) {
          idx++;
          const closeIdx = html.indexOf(quote, idx);
          if (closeIdx === -1) {
            value = html.slice(idx);
            idx = length;
          } else {
            value = html.slice(idx, closeIdx);
            idx = closeIdx + 1;
          }
        } else {
          const valueStart = idx;
          while (idx < length && !isWhitespace(html[idx]) && html[idx] !== ">") {
            idx++;
          }
          value = html.slice(valueStart, idx);
        }
      }
    }

    if (name && !hasOwn(attributes, name)) {
      attributes[name] = decodeEntities(value);
    }

    if (idx === loopStart) {
      idx++;
    }
  }

  return { attributes, selfClosing, idx };
}
```

Це твоя ж логіка, але вся вкладена в один зовнішній цикл. Замість `return;` стоїть `break`, а в кінці повертається об'єкт.

Крок 6. `parseOpenTag` і `parseCloseTag` (≈ 294–325): заміни обидві

```js
function parseOpenTag(html, idx) {
  const tagResult = readTagName(html, idx + 1);
  const attributeResult = readAttributes(html, tagResult.idx);

  return {
    element: createElement(tagResult.tagName, attributeResult.attributes),
    selfClosing: attributeResult.selfClosing,
    idx: attributeResult.idx,
  };
}
```

```
function parseCloseTag(html, idx) {
  const tagResult = readTagName(html, idx + 2);
  const closeIdx = html.indexOf(">", tagResult.idx);

  return {
    tagName: tagResult.tagName,
    idx: closeIdx === -1 ? html.length : closeIdx + 1,
  };
}
```

Крок 7. `parseComment` і `parseDeclaration` (≈ 327–368): заміни обидві
```js
function parseComment(html, idx) {
  const contentStart = idx + 4; // after "<!--"
  const end = html.indexOf("-->", contentStart);

  if (end === -1) {
    return { node: createComment(html.slice(contentStart)), idx: html.length };
  }
  return { node: createComment(html.slice(contentStart, end)), idx: end + 3 };
}

function parseDeclaration(html, idx) {
  const end = html.indexOf(">", idx);
  const nextIdx = end === -1 ? html.length : end + 1;
  const rawContent = html.slice(idx, nextIdx);

  const isDoctype = /^<!doctype/i.test(rawContent);
  const node = isDoctype ? createDoctype(rawContent) : createComment(rawContent);

  return { node, idx: nextIdx };
}
```
`console.error` прибрано. Поле тепер `idx`, як в усіх.

Крок 8. `readRawText` і `readText` (≈ 371–424): заміни обидві
```js
function readRawText(html, idx, tagName) {
  // tagName comes only from RAWTEXT_TAGS, so it is safe inside a RegExp
  const regex = new RegExp("</" + tagName + "(?=[\\s/>])", "gi");
  regex.lastIndex = idx;

  const match = regex.exec(html);
  const end = match ? match.index : html.length;

  let content = html.slice(idx, end);
  if (hasOwn(RCDATA_TAGS, tagName)) {
    content = decodeEntities(content);
```

```
  }

  return { content, idx: end };
}

function readText(html, idx) {
  let end = idx;

  while (end < html.length) {
    const nextAngle = html.indexOf("<", end);

    if (nextAngle === -1) {
      end = html.length;
      break;
    }

    const next = html[nextAngle + 1];
    const startsMarkup =
      isTagStart(html, nextAngle) ||
      isCloseTagStart(html, nextAngle) ||
      next === "!" ||
      next === "?";

    if (startsMarkup) {
      end = nextAngle;
      break;
    }

    end = nextAngle + 1;
  }

  return { content: decodeEntities(html.slice(idx, end)), idx: end };
}
```

Крок 9. Старий `html2json` унизу (≈ 427–438): видали і встав на це місце
```js
const MAX_DEPTH = 512;
const MAX_ERRORS = 100;

function hasOwn(obj, key) {
  return Object.prototype.hasOwnProperty.call(obj, key);
}

function reportError(errors, message) {
  if (errors.length < MAX_ERRORS) {
    errors.push(message);
  }
}
```

```
function prepareInput(value) {
  if (value === null || value === undefined) {
    return "";
  }
  const text = typeof value === "string" ? value : String(value);
  return text.charCodeAt(0) === 0xfeff ? text.slice(1) : text;
}

function isBlank(text) {
  for (let i = 0; i < text.length; i++) {
    if (!isWhitespace(text[i])) {
      return false;
    }
  }
  return true;
}

function closeElement(stack, tagName, errors) {
  for (let i = stack.length - 1; i > 0; i--) {
    if (stack[i].tag === tagName) {
      for (let j = stack.length - 1; j > i; j--) {
        reportError(errors, `<${stack[j].tag}> closed implicitly by </${tagName}>`);
      }
      stack.length = i;
      return;
    }
  }
  reportError(errors, `Stray closing tag </${tagName}>`);
}

function openElement(html, stack, parsed, errors) {
  const { element, selfClosing } = parsed;
  let idx = parsed.idx;

  stack[stack.length - 1].children.push(element);

  if (selfClosing || hasOwn(VOID_TAGS, element.tag)) {
    return idx;
  }

  if (stack.length >= MAX_DEPTH) {
    reportError(errors, `Maximum depth exceeded at <${element.tag}>`);
    return idx;
  }

  stack.push(element);

  if (hasOwn(RAWTEXT_TAGS, element.tag)) {
```

```
    const raw = readRawText(html, idx, element.tag);
    if (raw.content !== "") {
      element.children.push(createText(raw.content));
    }
    idx = raw.idx;
  }

  return idx;
}

function parseNext(html, idx, stack, errors) {
  const parent = stack[stack.length - 1];

  if (html.startsWith("<!--", idx)) {
    const result = parseComment(html, idx);
    parent.children.push(result.node);
    return result.idx;
  }

  if (html[idx] === "<" && (html[idx + 1] === "!" || html[idx + 1] === "?")) {
    const result = parseDeclaration(html, idx);
    parent.children.push(result.node);
    return result.idx;
  }

  if (isCloseTagStart(html, idx)) {
    const result = parseCloseTag(html, idx);
    closeElement(stack, result.tagName, errors);
    return result.idx;
  }

  if (isTagStart(html, idx)) {
    return openElement(html, stack, parseOpenTag(html, idx), errors);
  }

  const result = readText(html, idx);
  if (!isBlank(result.content)) {
    parent.children.push(createText(result.content));
  }
  return result.idx;
}

function html2json(htmlText) {
  const root = { type: "document", children: [], errors: [] };
  const stack = [root];

  try {
    const html = prepareInput(htmlText);
    let idx = 0;
```

```
    while (idx < html.length) {
      const nextIdx = parseNext(html, idx, stack, root.errors);
      idx = nextIdx > idx ? nextIdx : idx + 1;
    }

    for (let i = stack.length - 1; i > 0; i--) {
      reportError(root.errors, `Unclosed tag <${stack[i].tag}>`);
    }
  } catch (error) {
    reportError(root.errors, `Unexpected error: ${error.message}`);
  }

  return root;
}

if (typeof module !== "undefined" && module.exports) {
  module.exports = { html2json };
}
```
Останній блок `module.exports` потрібен лише щоб перевіряти код у Node. У браузері він нічого не робить.

Крок 10. Перевірка
У терміналі з папки проєкту:
```bash
node -e "const {html2json}=require('./html2json.js'); console.log(JSON.stringify(html2json('<div class=\"a\">Hi &amp;amp; bye<br><!-- c --></div>'),null,2))"
```
Очікуєш дерево з одним `div`, атрибутом `class: "a"`, текстом `Hi & bye`, елементом `br` і коментарем ` c `, а `errors` має бути порожнім. Потім перевір ще кілька ввідних: `""`, `"<div"`, `"<div><p>text</div>"`, `"a < b"`, `"<script>var s='</div>';</script>"`.

Після цього лишаються `showExample1`, `showExample2` і семпли. Напиши, коли заміниш усе, і я перевірю.

She wants exact locations and content to add, in chat rather than me editing the file directly. I'll lay out an ordered list by line number with the precise code for each change.

For the full rewritten functions like readAttributes, readTagName, and readText I'll just give the complete replacement block rather than trying to patch individual lines, since that's cleaner. I'm carefully recounting the actual line numbers in the displayed file to make sure my references to things like decodeEntities' numeric entity handling are accurate before I write up the list.

Writing decodeEntities logic...

Writing attribute parser logic...

Parsing attribute values...

I'm finishing the attribute loop, making sure to use hasOwnProperty checks for safety against prototype pollution, and verifying edge cases like attributes named "=" or empty names still advance the index properly so the parser never gets stuck.

Now I'm moving on to building the parseOpenTag function that ties together tag name reading and attribute reading...

Writing more parsing helpers...

Writing readRawText function...

Now I'm writing readText, which scans forward from idx looking for the next "<" that begins a real tag, closing tag, comment, or doctype, treating everything else as plain text content to decode and return.

Since function declarations hoist, the ordering doesn't matter even with MAX_DEPTH referenced before its definition appears. Next I should add example usage and a quick way to verify in Node, like running a one-liner that requires the module and logs the JSON output for a sample HTML string with attributes, entities, and comments.

I'll also move the MAX_DEPTH-style constants and the main block to the bottom with hasOwn hoisted there too, ensuring VOID_TAGS checks use hasOwn safely against prototype pollution. Now I'll write the final answer in Ukrainian, using markdown code blocks only for the new code.

КОРИСТУВАЧ

Thursday, Oct 29, 2026, 9:42 PM (UTC+2)

я ж видалила ці 2 функції, мені потрібно замість них в index html поставити інші?

АСИСТЕНТ (CURSOR)

I'm trying to figure out which two functions she deleted — possibly showExample1/showExample2, which index.html's buttons reference but may not exist in the project. I should check the actual file and index.html to confirm.

Ні, `index.html` міняти не треба. Не впевнений, які саме дві функції ти маєш на увазі. Найімовірніше, це `showExample1` і `showExample2`, тож відповідаю про них. Якщо ти мала на увазі інші, напиши назви.

У index.html на рядках 64–65 кнопки викликають ці функції:

```html
<button onclick="showExample1()">Input Example 1</button>
<button onclick="showExample2()">Input Example 2</button>
```

У проєкті цих функцій немає. У початковому `html2json.js` їх теж не було, тож видалити їх ти не могла. Зараз натиснення цих кнопок дає `ReferenceError: showExample1 is not defined`. Рядки в HTML лишаються як є, а самі функції треба дописати в `html2json.js`, поруч із `convertHtml2JsonAndSet`.

Що вони мають робити: покласти приклад HTML у поле `#html`. Кнопка «Convert to JSON» перетворить його як звичайний ввід.

Приклад, який можна вставити, наприклад, одразу після `convertHtml2JsonAndSet` на початку файлу:

```js
function showExample1() {
  document.getElementById("html").value =
`<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Test &amp;amp; example</title>
  </head>
  <body>
    <!-- main block -->
    <div class="box" id="main">
      <p>Hello &amp;copy; world<br>second line</p>
      <img src="a.png" alt="pic">
    </div>
  </body>
</html>`;
}

function showExample2() {
  document.getElementById("html").value =
`<div class="a" data-x=1 hidden>
  <p>unclosed paragraph
  <span>text</div>
</b>
<a href="broken`;
}
```

Перший приклад показує нормальну структуру: doctype, атрибути, коментар, сутності, void-теги. Другий навмисно битий: незакритий `p`, зайвий `</b>`, обірваний атрибут. Так видно, що парсер не падає на поганому вводі.

Ще одне: `convertHtml2JsonAndSet` на самому початку файлу вже використовує `JSON.stringify` без захисту. Напиши, коли дойдеш до задачі 6 (`try/catch` навколо нього), і я перевірю.

I'm trying to figure out what she means by "deleting those 2 functions" — probably showExample1/showExample2, which may be referenced by buttons in index.html but no longer exist in the code, so she's asking whether she needs to replace those calls with something else.

My answer: she doesn't need to change index.html — the buttons should still call showExample1/2, but those functions need to actually be defined in html2json.js since they're missing, otherwise clicking throws a ReferenceError. I'll note the ambiguity but give her a short concrete fix.

КОРИСТУВАЧ

Thursday, Oct 29, 2026, 10:37 PM (UTC+2)

Я дописала головну функцію.

Переглянь увесь html2json.js як сіньйор-рев'юер: що може зламатися на валідному HTML?» має знайтися type: "content"

АСИСТЕНТ (CURSOR)

Перегляну весь `html2json.js` як рев’юер: шукаю місця, де валідний HTML дає неправильне дерево або падіння.

Запущу парсер на кількох валідних фрагментах, щоб рев’ю спиралось на реальний результат, а не лише на читання коду.

`createComment` у `html2json.js` повертає коментар із `type: "content"` і полем `comment`. Для валідного `<!-- hi -->` виходить `{ "type": "content", "comment": " hi " }`, без помилок у `errors`. Текст і doctype лежать у полі `content`, коментар — ні, тож за типом його не знайти.

```133:138:html2json.js
function createComment(comment) {
  return {
    type: "content",
    comment,
  }
}
```

Тип має бути `"comment"`, текст — у полі `content`, як у `createText` і `createDoctype`. Той самий об’єкт іде і з `parseComment`, і з `parseDeclaration` для всього, що не doctype (`<?xml ...?>`).

На звичайній розмітці парсер не падає. Я прогнав кілька фрагментів: `<title>A & B</title>` дає текст `A & B`, `<br>` лишається порожнім елементом усередині `<p>`, `<script>` з рядком `"</div>"` не закривається завчасно, `<!DOCTYPE html>` стає окремим вузлом.

Далі дерево на валідному HTML розходиться з тим, як його читає браузер.

Пробіли між тегами зникають. `isBlank` у `parseNext` викидає текстовий вузол, якщо в ньому лише пробіли. `<span>a</span> <span>b</span>` перетворюється на два `span` підряд, пробіл між ними втрачено. `<pre>` з самими переносами рядка виходить з порожнім `children`. У `<pre>` ці пробіли є вмістом.

Необов’язкові закривні теги не закриваються. У HTML `<li>` закриває попередній `<li>`, а `<h1>` (і `div`, `ul`, `table`) закриває відкритий `<p>`. Зараз цього немає:

- `<ul><li>one<li>two</ul>` кладе другий `li` всередину першого;
- `<p>text<h1>title</h1>` кладе `h1` всередину `p` і лишає `p` незакритим.

На реальній сторінці з купою `<li>`, `<p>`, `<td>` без `</...>` структура буде вкладена не так, як у

браузері. Код при цьому не впаде.

`<div/>` закриває не-void тег. Для `<div/>text</div>` виходить порожній `div`, текст `text` поруч і помилка `Stray closing tag </div>`. У HTML слеш у такому тегу ігнорується: `div` лишається відкритим, `text` — його дитина. Для `<br/>` і `<img/>` поточна поведінка правильна, бо вони й так у `VOID_TAGS`.

Рядок `"</script>"` усередині `<script>`. Регекс у `readRawText` зупиняється на першому `</script`, навіть якщо це лапки в JavaScript. Решта документа стає сусідами скрипта. Це відома межа такого сканера, її варто прямо описати в README.

`showExample1` і `showExample2` на місці, `index.html` чіпати не треба.

КОРИСТУВАЧ

Thursday, Oct 29, 2026, 10:47 PM (UTC+2)

```/Users/admin/Desktop/NEXT/jito-intern-test-task/html2json.js:463:465
if (!isBlank(result.content)) {
    parent.children.push(createText(result.content));
  }
```

перевірку тут прибрати, мається на увазі прибрати?

АСИСТЕНТ (CURSOR)

Так. Прибери умову `if (!isBlank(...))` і завжди додавай текстовий вузол, навіть якщо в ньому лише пробіли або переноси рядка.

```js
parent.children.push(createText(result.content));
```

Після цього `<span>a</span> <span>b</span>` збереже пробіл між тегами, а `<pre>` з самими переносами рядка не буде порожнім. Функція `isBlank` більше ніде не викликається, тож її теж можна видалити.

КОРИСТУВАЧ

Thursday, Oct 29, 2026, 11:13 PM (UTC+2)

такс, а як би захендлить отаку абру-кадабру?

"<<<<<"

АСИСТЕНТ (CURSOR)

Перевірю, що парсер зараз повертає на `"<<<<<"`, і від цього скажу, чи треба щось міняти.

`"<<<<<"` вже обробляється правильно, і окремий випадок для нього не потрібен. Парсер повертає один текстовий вузол і порожній масив `errors`:

```json
{ "type": "document", "children": [{ "type": "text", "content": "<<<<<" }], "errors": [] }
```

У `readText` символ `<` стає початком розмітки лише тоді, коли далі йде літера (`<div`), `/` і літера (`</div`), `!` або `?`. У `"<<<<<"` після кожного `<` знову стоїть `<`, тож жоден із них не є тегом. Функція зсувається на один символ і врешті забирає весь рядок як текст. Цикл у `html2json` теж захищений: якщо позиція не зрушилася, вона збільшується на 1, тож зависання не буде.

Те саме відбувається з `"a < b"` і `"<>"`: обидва лишаються текстом. А `"<<div"` ділиться навпіл, бо другий `<` уже справжній тег: текст `"<"`, елемент `div` і запис `Unclosed tag <div>` в `errors`.

КОРИСТУВАЧ

Thursday, Oct 29, 2026, 11:15 PM (UTC+2)

а може краще б вивести в масив "errors"?

АСИСТЕНТ (CURSOR)

Ні. `"<<<<<"` лишай текстом, в `errors` його не клади. У HTML символ `<`, після якого немає імені тега, це звичайний текст, і браузер покаже його так само. Рядки на кшталт `a < b` або `x < 5` зустрічаються в нормальних сторінках, і якби кожен такий `<` потрапляв в `errors`, валідний HTML виглядав би як помилка.

`errors` варто лишати для випадків, де дерево довелося відновити: незакритий тег, зайвий `</div>`, неявне закриття. У `"<<<<<"` відновлювати нічого, дерево правильне. У `"<<div"` помилка вже є, але лише на справжньому тегу: `Unclosed tag <div>`. Перший `<` при цьому залишається текстом.

КОРИСТУВАЧ

Thursday, Oct 29, 2026, 11:20 PM (UTC+2)

такс, я прогнала тестові файли в папці html_samples, наче ок, як на мене
переглянь ще ти...

АСИСТЕНТ (CURSOR)

Прогоню всі файли з `html_samples` через поточний парсер і подивлюсь, де дерево або `errors` розходяться з очікуваним.

Жоден файл не впав. `valid-small.html` і `rozetka.html` (256 КБ, приблизно 3 мс, 1467 вузлів, `errors` порожній) розібрані правильно. У `table.html` і `broken.html` дерево не те, яке здається з вигляду файлів.

`broken.html` майже нічого не перевіряє. Лапки в `<a href="broken` не закриті, тому все після них стає значенням атрибута `href`, а не розміткою. У дереві лише чотири дитини: `div`, два текстові переноси і `a`. Рядки з `<ul><li>`, `<p><h1>`, `<div/>` і `<<<<<` парсер як теги не бачить. Закрий лапки на цьому рядку (`<a href="broken">`) або перенеси його в кінець файлу. Після цього знову проганяй і перевір, що в дереві з'явились `ul`, `h1` і текст `<<<<<`.

`table.html` не падає, але вкладеність не браузерна. Це саме те обмеження, яке вже записане в

README. `<th>Name<th>Qty` кладе другий `th` всередину першого, `<td>Apple<td>2` так само, другий `<tr>` опиняється всередині `th`, другий `<option>` всередині першого. Десять записів у `errors` з'являються лише в кінці, коли `</table>` і `</select>` закривають одразу весь стек. Вони не фіксують момент, де наступний `td` мав закрити попередній.

`raw-text.html` спрацьовує так, як написаний парсер. `style` зберігає `.box > span`, `textarea` декодує `&` і не робить з `<b>` тег, `pre` зберігає пробіли, перший `script` залишає рядок `"</div>"` усередині себе. Другий `script` обривається на рядку `"</script>"`: текст `console.log("this is outside the script")` стає звичайним текстом у `body`, а справжній `</script>` потрапляє в `errors` як зайвий. Третій `script` із `<\/script>` залишається цілим. Це очікувана межа, не падіння.

У `valid-small.html` коментар має тип `"comment"`, але текст лежить у полі `comment`, тоді як у тексту і doctype поле називається `content`. На результат прогону це не впливає.

КОРИСТУВАЧ

Thursday, Oct 29, 2026, 11:26 PM (UTC+2)

згенеруй мені пдф файл з контенту цього чату

АСИСТЕНТ (CURSOR)

Зроблю PDF із цього діалогу й покладу його в `ai_help`, бо саме таку теку завдання просить для матеріалів про використання ШІ. Спочатку зберу текст розмови й перевірю, чим на цьому комп’ютері можна зібрати PDF.